

LipiSetu: Script-Invariant Hindi Search

One question, every script

CSD358 Information Retrieval | Mid-term Hackathon 2026 | Track T5: Multilingual and Indic-language Search

Name	Roll No.	Main contribution
Devansh Pandey	2310110461	Design, IR engine, Dhvani key, neural parts, evaluation, demo, report
Anamika Pal	2310110037	Human query annotation, annotator-agreement analysis
Abhinav Bachchas	2310110383	Human query annotation, results review, report proofreading

Code: github.com/devansh101005/IR-Hackathon Demo video: [Google Drive link \(7:52\)](#)

Abstract

Most people type Hindi in Roman letters, and everyone spells it differently (*mausam*, *mosam*, *mousam*). Most Hindi text on the web, however, is written in Devanagari (मौसम). A normal search engine matches exact words, so these users get much worse results. On 350 judged MIRACL-Hindi test queries, BM25 reaches $nDCG@10$ 0.533 for questions typed in Devanagari, but only 0.004 for the same questions typed in Roman letters, and the obvious fix (transliterating the query first) only reaches 0.113. We call this loss the **Script Gap**. LipiSetu closes most of it with classic IR, written from scratch: a sound-based key for every word (the **Dhvani key**) stored as its own index zone, **pooled document frequency** across spellings, and query-term proximity inside BM25. With these, Roman queries reach 0.487. A small distilled neural encoder (448 MB $\rightarrow$ 37 MB) runs only when a gate trained on IR signals predicts it will help. A human query set typed by our three team members confirms the results on real spellings.

1. Problem and Track Relevance

1.1 The user need

On phones, Hindi is mostly typed in Roman letters ("kal ka mausam"), and every person spells words in their own way. But most content people search for (Wikipedia, news, government pages) is written in Devanagari ("कल का मौसम"). An inverted index matches exact terms, so a Roman query finds almost nothing, even when the answer is in the index. The mixing also happens inside the data: 19% of the MIRACL-Hindi passages we index already contain Roman-script words. Our goal is simple to state: **the same question should get the same results, whatever script it is typed in.**

1.2 Why it belongs to Track T5

Track T5 asks for search in an Indian language with code-mixed or transliterated queries. Our system covers every IR topic listed for the track:

- Tokenisation and normalisation for a non-English script (Devanagari).
- A Hindi stemmer, compared with no stemming.
- Soundex-style phonetic matching for transliterated spellings (our Dhvani key, compared with Soundex).
- How stop words and idf behave on a Hindi corpus.
- Cross-lingual ranking in a vector space (tf-idf cosine and a multilingual embedding model).

1.3 Papers referred

Mixed-script retrieval was studied by Gupta et al. [2] and in the FIRE 2015 shared task [3]; we use the MIRACL dataset [1] for judged Hindi queries. Pooled df is based on Pirkola's structured queries [4]; our consistency metric uses Rank-Biased Overlap [5]; BM25 follows Robertson and Zaragoza [6]; the distillation follows Reimers and Gurevych [11]; the gate uses ideas from query performance prediction [8] and cascade ranking [9]. The course textbook [15] is the reference for all classic IR parts.

2. How We Used IR

Figure 1 shows the whole pipeline. The left half is built once (offline); the right half runs for every query. Each box names the file that implements it, so every IR idea can be traced to the code.

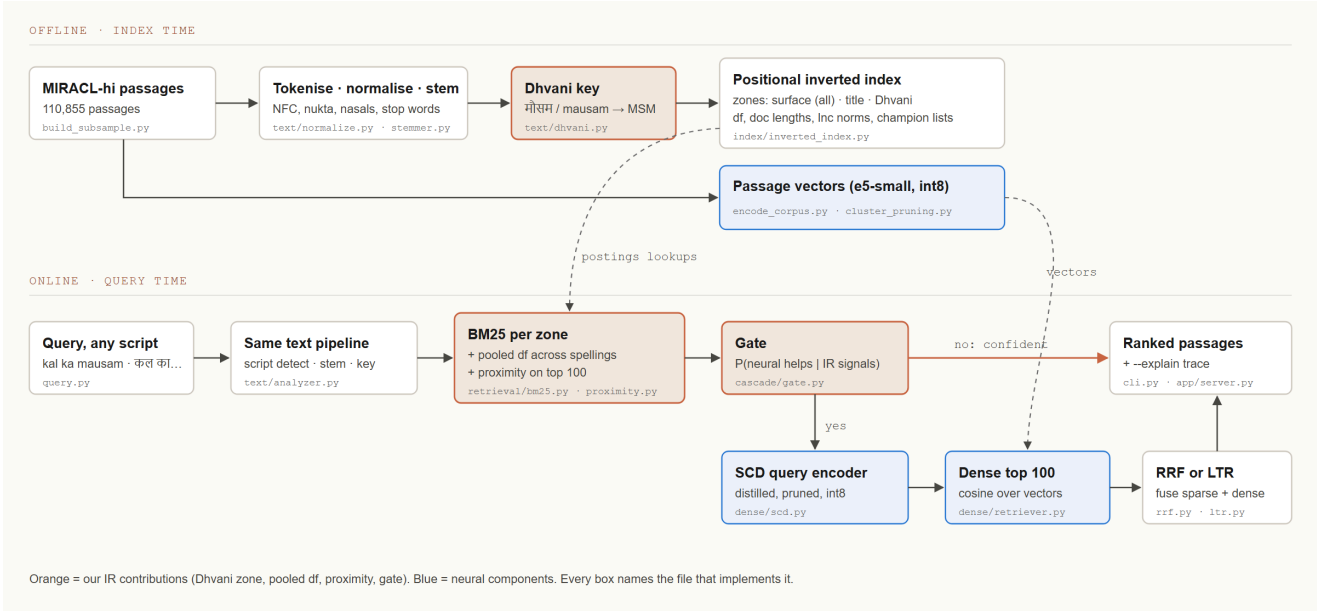


Figure 1: LipiSetu pipeline. Orange boxes are our IR contributions, blue boxes are neural components.

2.1 Documents and text processing

Documents. One MIRACL passage is one document, because the relevance labels are given per passage. Each document has a *title* and a *body* zone. We index every passage judged for any query plus a fixed random sample: 110,855 passages and 8,081,081 tokens.

Tokenisation and normalisation (`text/tokenize.py`, `normalize.py`). Python's `\w` does not match Devanagari vowel signs, so a normal regex breaks हिन्दी into three pieces. We tokenise with explicit Unicode ranges instead. Normalisation applies Unicode NFC, folds nukta letters (क़ → क), maps chandrabindu to anusvara, removes invisible joiners, converts Devanagari digits, and lower-cases Roman text.

Stop words and idf. Hindi stop words are very frequent postpositions and helper verbs; the most frequent terms in our corpus are के, में, है, की, का, और. 35 of the 40 highest-df terms are in our stop list (Hindi, Hinglish and English). The corpus follows Zipf's law (Appendix G), and 151,995 of the 274,129 vocabulary terms occur only once.

Stemming (`text/stemmer.py`). A light suffix stripper, after Ramanathan and Rao [13], maps लड़का / लड़के / लड़कों to one stem. It raises Devanagari nDCG@10 from 0.492 (no stemming) to 0.533.

2.2 The Dhvani key (our phonetic key)

Soundex gives words that sound alike the same code, but it only works for English letters and cuts codes to four characters. Our Dhvani key (`text/dhvani.py`) works on **both scripts**: each consonant sound in Devanagari or Roman maps to one letter class, and the rules in Table 1 remove the differences that Roman spelling cannot show. मौसम, mausam, mosam and mousam all become MSM. We do not cut the key to four characters, because Hindi words are short and cutting merges too many different words. Classic Soundex is also implemented, as a baseline.

Table 1: Dhvani key rules

Rule	Example	Why
Aspirated = unaspirated	kh = k, bh = b (ख = क)	people often skip the h
Retroflex = dental	ट = त, ड = द	Roman letters cannot show the difference
Drop vowels (keep a first vowel as A)	mausam, mosam → MSM	vowels are spelled in many ways
w = v, z = j, f = ph	zindagi = jindagi	common spelling swaps
Nasal before a consonant → N	samvidhan = संविधान	anusvara has no Roman letter
Collapse repeated sounds	pakka = पक्का → PK	double letters are optional

2.3 Inverted index with zones

The index (`index/inverted_index.py`) is a dictionary plus postings lists with positions, stored in numpy arrays. It has three main zones: *all* (stemmed words of title and body), *title*, and *dhvani* (the key of every word). Extra zones (*soundex*, *nostem*, *raw*) exist only for the baselines. Boolean AND/OR/NOT processes terms in order of increasing df and uses skip pointers; phrase queries use the positions (`retrieval/boolean.py`).

2.4 Scoring

Each zone is scored with BM25 (`retrieval/bm25.py`), and the zone scores are added with weights:

$$\text{score}(d) = \text{BM25}_{\text{all}}(d) + w_{\text{dhvani}} \cdot \text{BM25}_{\text{dhvani}}(d) + w_{\text{title}} \cdot \text{BM25}_{\text{title}}(d) + \lambda \cdot (m - 1) / (\text{window} - 1) \quad (1)$$

The last term is **query-term proximity** (`proximity.py`): the smallest window of word positions that contains all m matched query words. It is applied to the top 100 results. **Pooled df** fixes a subtle problem: one word can have many spellings, and each spelling alone looks rare, so its idf becomes too high. With pooled df, every spelling uses the df of its whole Dhvani class (all documents that contain any spelling of the word). This is Pirkola's idea [4], applied to spellings instead of translations. Scores are accumulated term-at-a-time and the top K are taken with a heap (`topk.py`). We also implement tf-idf (Inc.ltc cosine) as the vector-space comparison. All parameters were tuned on the **train split only**: $k1 = 0.9$, $b = 0.5$, $w_{\text{dhvani}} = 2.0$, $w_{\text{title}} = 1.0$, $\lambda = 2.0$.

Table 2: IR concepts from the course and where they are in our code

Lecture topic	What we implemented	File (src/lipisetu/)
Boolean retrieval	AND/OR/NOT, df-ordered merge, skip pointers, phrase queries	retrieval/boolean.py
Term vocabulary	Tokeniser, normaliser, stop words, stemmer, Soundex, Dhvani key	text/
Index construction	Positional inverted index with zones, Inc norms	index/inverted_index.py
tf-idf and VSM	log tf, idf, Inc.ltc cosine	retrieval/tfidf.py
Scoring	BM25 per zone, zone weights, pooled df	retrieval/bm25.py
Result assembly	Heap top-K, champion lists, query-term proximity	retrieval/topk.py, proximity.py
Static quality $g(d)$	Lead-passage prior used as a ranking feature	rerank/ltr.py
Cluster pruning	$\sqrt{N}$ leaders, followers of the nearest leaders scored	dense/cluster_pruning.py
Evaluation	P@k, Recall@k, MRR, nDCG, RBO, significance test	eval/metrics.py

3. Beyond IR

Dense retrieval (`dense/`). Every passage is turned into a vector once with the multilingual-e5-small model [14], exported to ONNX and compressed to int8 so it runs fast on a laptop CPU. A query is answered with a dot product over all passage vectors, or approximately with cluster pruning from the lectures.

Script-consistency distillation (SCD). We train a copy of the encoder (the student) so that a Roman spelling of a question lands at the same point as the original model's vector for the Devanagari question (loss = $1 - \text{cosine}$), following [11]. Only the query side changes, so the passage vectors are reused. On held-out train questions, the cosine between a Roman question and its Devanagari target rises from 0.783 to 0.902. We then keep only the 43,366 tokens our data uses (out of 250,002) [12] and compress to int8: 448 MB $\rightarrow$ 37 MB.

Learning to rank (`rerank/ltr.py`). A logistic regression re-orders the candidates using IR features: BM25 per zone, proximity, tf-idf, the dense score, ranks in both lists, the static quality prior and passage length [10]. It is trained on the train judgments only.

Gated cascade (`cascade/gate.py`). The neural stage is slow, and it does not always help. The gate looks at signals the fast search gives for free (top score, score gap, NQC [8], how many words matched only through the Dhvani zone, max idf, query length, script) and predicts whether running the neural stage will improve the ranking. Its threshold is chosen on the train split.

4. Novelty and Creativity

The obvious T5 project, and the sample idea in the assignment, is a Hinglish search engine that transliterates the query and ranks it with BM25. In our project that is only the baseline (B2). Table 3 lists what we do differently.

Table 3: The obvious approach compared with LipiSetu

Aspect	Obvious approach	LipiSetu
Evaluation	Average nDCG or P@k only	Script Gap, cross-script consistency (RBO), worst-script nDCG
Roman queries	Transliterate the query, then BM25	Dhvani zone inside the index, no transliteration model
df of spelling variants	Each spelling has its own df	Pooled df across spellings
Neural stage	Always on, or not used	Runs only when an IR-signal gate says it helps
Model size	Off-the-shelf encoder	Distilled for script consistency, pruned, int8 (37 MB)
Test queries	Generated variants only	Also 60 questions romanised by 3 people independently

Prior work. Mixed-script search is not new [2, 3], and Soundex variants for Indian languages exist. We do not claim to be the first. To our knowledge, what is new for Hindi on MIRACL is the combination: measuring script invariance directly, a cross-script phonetic zone with pooled df inside a classic BM25 engine (with a rule-by-rule ablation), and neural inference that is gated by IR signals and reported as a quality-versus-compute trade-off.

5. Evaluation

5.1 Setup

Queries and judgments. The test set is the 350 MIRACL-Hindi dev queries (3,494 judged passages, about 2.1 relevant passages per query). All tuning and training uses the separate 1,169 train queries only. Every test query is asked in three forms: **F1** the original Devanagari, **F2** a standard Roman spelling, and **F3** a casual Roman spelling with random spelling noise (`text/romanize.py`). Unjudged passages count as not relevant.

Metrics. nDCG@10 is the main metric. P@10 is reported too, but with only about 2 relevant passages per query it cannot go above about 0.2. To measure script invariance we add three metrics:

- **Script Gap** = nDCG@10 for Devanagari minus nDCG@10 for a Roman form (0 = script does not matter).
- **CSC@10** (cross-script consistency) = average Rank-Biased Overlap [5] between the top-10 lists of the three forms (1 = identical rankings).
- **Worst-script nDCG@10** = for each query, the nDCG@10 of its worst form, averaged.

Systems. B0 raw BM25; B1 BM25 with normalisation and stemming; B1N without stemming; B2 transliteration + BM25 (the obvious baseline); V1 tf-idf cosine; S0 B1 + Soundex zone; S1 B1 + Dhvani zone; S2 S1 + pooled df; S3 S2 + title zone + proximity; D0 dense e5-small; D1 our distilled encoder; H1 S3 + D1 fused with RRF [7]; L1 learning to rank; G1 the gated cascade.

5.2 Main results

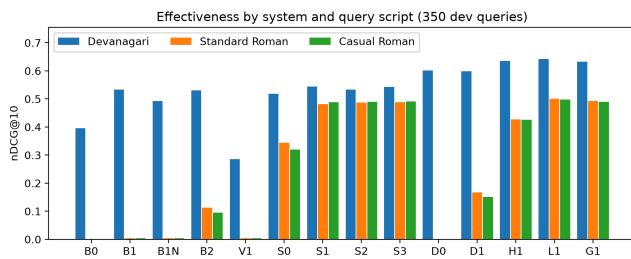


Figure 2: nDCG@10 by system and query script.

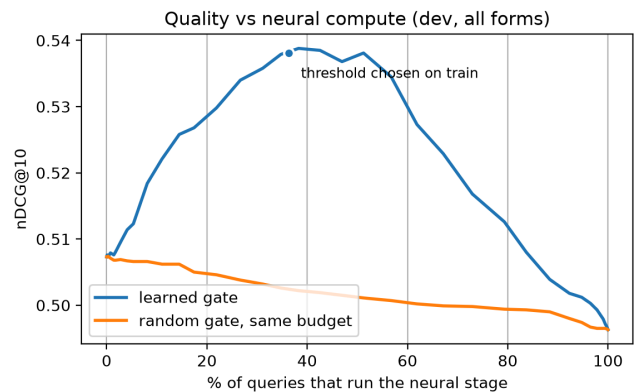


Figure 3: Quality against the share of queries that run the neural stage.

Table 4: Search quality on the 350 test queries (bold = our main systems)

	system	nDCG@10 Deva	nDCG@10 Roman	nDCG@10 casual	P@5 Deva	R@100 Deva	R@100 casual
B0	BM25, raw whitespace tokens	0.395	0.000	0.000	0.173	0.785	0.006
B1	BM25 + normalisation + stemming	0.533	0.004	0.003	0.231	0.937	0.012
B1N	B1 without stemming	0.492	0.004	0.003	0.217	0.903	0.012
B2	B1 + Roman→Devanagari transliteration	0.531	0.113	0.095	0.230	0.937	0.334
V1	tf-idf lnc.ltc cosine	0.286	0.004	0.004	0.127	0.833	0.009
S0	B1 + classic Soundex zone	0.518	0.344	0.320	0.219	0.915	0.698
S1	B1 + Dhvani zone	0.544	0.481	0.488	0.239	0.933	0.865
S2	S1 + pooled df	0.533	0.487	0.490	0.234	0.926	0.865
S3	S2 + title zone + proximity	0.543	0.488	0.491	0.246	0.932	0.865
D0	dense e5-small (int8)	0.601	0.001	0.000	0.258	0.954	0.007
D1	dense, SCD query encoder	0.598	0.167	0.151	0.254	0.964	0.391
H1	RRF(S3, D1)	0.636	0.427	0.426	0.279	0.981	0.865
L1	learning to rank	0.642	0.501	0.498	0.288	0.993	0.895
G1	gated cascade	0.633	0.492	0.490	0.277	0.977	0.871

The Script Gap is real and large. BM25 with stemming (B1) reaches 0.533 on Devanagari questions but 0.004 on the same questions in Roman. Transliterating the query (B2) only recovers 0.113 / 0.095 (standard / casual Roman), because casual spellings are not valid transliteration input: *bharat* becomes भरत (a name), not भारत (India). Classic Soundex (S0) helps (0.344) but its four-character English codes merge too many words. The Dhvani zone (S1) reaches 0.481 / 0.488, with almost no loss on Devanagari.

Table 5: Script invariance (lower Script Gap and higher CSC are better)

	system	Script Gap Roman	Script Gap casual	CSC@10 (RBO)	worst-script nDCG@10
B0	BM25, raw whitespace tokens	0.395	0.395	0.228	0.000
B1	BM25 + normalisation + stemming	0.529	0.530	0.179	0.002
B1N	B1 without stemming	0.488	0.489	0.179	0.003
B2	B1 + Roman→Devanagari transliteration	0.418	0.436	0.244	0.066
V1	tf-idf lnc.ltc cosine	0.281	0.282	0.185	0.002
S0	B1 + classic Soundex zone	0.174	0.198	0.534	0.296
S1	B1 + Dhvani zone	0.063	0.057	0.774	0.452
S2	S1 + pooled df	0.046	0.043	0.794	0.456
S3	S2 + title zone + proximity	0.054	0.051	0.748	0.444
D0	dense e5-small (int8)	0.600	0.600	0.169	0.000
D1	dense, SCD query encoder	0.432	0.447	0.299	0.127
H1	RRF(S3, D1)	0.209	0.210	0.508	0.353
L1	learning to rank	0.141	0.143	0.637	0.456
G1	gated cascade	0.141	0.143	0.625	0.432

With pooled df (S2), the Script Gap on casual Roman falls from 0.436 (B2) to 0.043, and cross-script consistency rises from 0.244 to 0.794. In other words, LipiSetu gives almost the same answer whichever script the question is typed in.

5.3 Neural models and the gate

Neural models have their own script problem. The plain dense model (D0) is the best single model on Devanagari (0.601) but scores 0.000 on casual Roman: for a Roman question it finds other Roman-script passages, so it matches

the script before the meaning. Our distillation (D1) fixes part of this (0.151) at almost no cost on Devanagari (0.598), but stays far below the Dhvani zone on Roman. Simple fusion (H1) is best on Devanagari (0.636) but pulls Roman down to 0.427. Learning to rank (L1) uses both signals and is the best system on every script (0.642 / 0.501 / 0.498), with the highest worst-script nDCG (0.456 against 0.066 for B2).

The gate spends neural compute only where it helps. Because the neural stage helps Devanagari questions but hurts many Roman ones, always running it (0.496) is worse than never running it (0.507). With its threshold chosen on train, the gate runs the neural stage for 36% of queries and reaches 0.538, better than both, and clearly above a random gate with the same budget (Figure 3). Its median latency is 18.6 ms, against 23.8 ms for always running the neural stage.

tf-idf against BM25. Inc.ltc cosine (V1) scores 0.286 against 0.533 for BM25. Full cosine normalisation favours short passages: the median length of V1's top result is 12 terms, against 35 for the corpus and 57 for BM25.

5.4 Human query set

Our Roman test forms are generated by a program, so we also built a small test set from real typing. Each of the three team members took the same 60 test questions and typed them in Roman letters on their own, without any transliteration tool and without looking at each other's sheets. Each member also wrote code-mixed and English versions of 20 questions. All three spelled a word identically for only 78% of the words, but their Dhvani keys were the same for 91% of the words.

Table 6: nDCG@10 on the human query set (60 questions; A1–A3 are the three annotators)

	system	Deva	Roman A1	Roman A2	Roman A3	code-mixed	English
B1	BM25 + normalisation + stemming	0.571	0.020	0.019	0.013	0.033	0.015
B2	B1 + Roman→Devanagari transliteration	0.571	0.140	0.112	0.117	0.054	0.039
S0	B1 + classic Soundex zone	0.568	0.270	0.270	0.253	0.100	0.051
S3	S2 + title zone + proximity	0.616	0.448	0.435	0.438	0.204	0.119
D0	dense e5-small (int8)	0.611	0.005	0.005	0.005	0.043	0.346
D1	dense, SCD query encoder	0.595	0.188	0.208	0.179	0.316	0.413
H1	RRF(S3, D1)	0.697	0.437	0.404	0.433	0.341	0.333
L1	learning to rank	0.706	0.507	0.507	0.491	0.383	0.369
G1	gated cascade	0.691	0.454	0.438	0.434	0.260	0.258

On the real romanisations BM25 reaches only 0.013–0.020 and transliteration 0.112–0.140, while LipiSetu (S3) reaches 0.435–0.448 ($p \leq 0.0002$ against B2 for every annotator) and learning to rank 0.491–0.507. So the results on generated queries hold for real spelling variation. Code-mixed and English questions are different: a sound key cannot link *leader* to नेता, so S3 drops to 0.204 (code-mixed) and 0.119 (English), and only the neural encoder helps (D1 0.413 on English). The gate never saw such questions in training and calls the neural stage too rarely for them (G1 0.258 against H1 0.333 on English, $p = 0.0009$).

5.5 Which parts matter

Table 7: Dhvani rule ablation (one rule switched off, Dhvani zone rebuilt)

variant	keys	words/key	nDCG Deva	nDCG Roman	nDCG casual	CSC@10
all rules (Dhvani)	100,839	2.65	0.544	0.481	0.488	0.774
without merge_aspiration	111,942	2.39	0.547	0.494	0.479	0.760
without merge_retroflex	111,891	2.39	0.546	0.382	0.388	0.651
without drop_vowels	219,235	1.23	0.518	0.166	0.123	0.319
without merge_wv	101,689	2.63	0.544	0.481	0.480	0.763
without merge_zj	101,232	2.64	0.544	0.479	0.482	0.755
without merge_fp	101,841	2.62	0.544	0.480	0.486	0.771
without nasal_rule	101,007	2.64	0.544	0.472	0.478	0.767
without collapse_doubles	113,486	2.36	0.542	0.484	0.488	0.788

Ablation. Dropping vowels is by far the most important rule, followed by merging retroflex and dental sounds (Table 7). Collapsing doubled letters slightly hurts; we kept the rule set as designed instead of tuning it on the test

queries.

Table 8: Pooled df on a mixed-script corpus (30% of passages romanised)

	system	nDCG Deva	nDCG Roman	nDCG casual
B1	BM25 + normalisation + stemming	0.416	0.201	0.150
B2	B1 + Roman→Devanagari transliteration	0.415	0.106	0.089
S1	B1 + Dhvani zone	0.460	0.314	0.351
S2	S1 + pooled df	0.465	0.358	0.390
S3	S2 + title zone + proximity	0.464	0.332	0.375

Pooled df. On the Devanagari-only corpus, pooled df trades a little Devanagari accuracy (0.544 → 0.533) for Roman accuracy and consistency. On a corpus where 30% of passages are romanised, a word's df really is split between its spellings, and pooled df helps every form: standard Roman 0.314 → 0.358, casual Roman 0.351 → 0.390, consistency 0.346 → 0.408 (idf examples in Appendix F).

5.6 Efficiency and significance

Efficiency (Appendix A and E, laptop CPU, no GPU). The sparse engine S3 answers in 14.0 ms (median). Skip pointers halve the comparisons in AND merges. Champion lists are 5–6 times faster but lose a lot of quality. Cluster pruning scores only about 5% of the passage vectors and keeps half of the exact top 100, but on our laptop it is not faster, because one numpy dot product over 110k vectors already takes only a few milliseconds.

Significance (Appendix D). Paired randomisation tests (10,000 permutations, nDCG@10) give $p < 0.001$ for every Roman-query gain discussed above (S1 over B1 and S0, S3 over B2, D1 over D0, L1 and G1 over H1). On Devanagari questions the sparse systems are not significantly different from B1/B2, so the gains on Roman come without a loss on Devanagari.

6. Limitations and Next Steps

- **Sound keys merge some different words.** *kal* (कल, tomorrow) and *khel* (खेल, game) both become KL, so sports passages can appear for a weather question (shown live in our video). Zone weights reduce this but do not remove it.
- **Meaning, not sound.** English words in code-mixed questions (*leader* for नेता) are only matched by the neural stage, and the gate was not trained on such questions.
- **Subsampled corpus.** We index a subsample of MIRACL-Hindi, so our scores are not comparable to the MIRACL leaderboard.
- **Small human set.** 60 questions; one annotator (Devansh) also designed the Dhvani key, so his romanisation is not blind.
- **Hindi only.**

Roadmap (if continued as the course project): (1) Bengali and Telugu from MIRACL with their own Dhvani tables; (2) a larger human query set, including voice queries; (3) the int8 encoder running in the browser, so search works fully on a phone; (4) learning the key rules from data instead of writing them by hand.

7. Work Division

Table 9: Who worked on what

Member	Contribution
Devansh Pandey 2310110461	Idea and system design; text pipeline, inverted index, Boolean and phrase search, tf-idf, BM25, proximity, top-K and champion lists; the Dhvani key and pooled df; dense retrieval, distillation, vocabulary pruning, cluster pruning; learning to rank; the gated cascade; evaluation framework and metrics; demo; report; one of the three annotators (code-mixed and English for rows 1–20).
Anamika Pal 2310110037	Annotated the human query set (60 romanised questions; code-mixed and English for rows 21–40); analysed the annotator-agreement results and presents them in the video.
Abhinav Bachchas 2310110383	Annotated the human query set (60 romanised questions; code-mixed and English for rows 41–60); reviewed the final results and plots; proofread the report; presents the evaluation and the live limitation in the video.

AI-Use Declaration

We used Claude Code (an AI coding agent by Anthropic) to write code, tests and documentation and to run the experiments, under Devansh's direction. Devansh chose the track and the idea, designed the system, reviewed the code and checked every result. The human query annotations were written by the three of us without AI tools. Every number in this report was produced by the scripts in the repository on the real data.

References

1. X. Zhang et al. MIRACL: A Multilingual Retrieval Dataset Covering 18 Diverse Languages. TACL, 2023.
2. P. Gupta, K. Bali, R. E. Banchs, M. Choudhury, P. Rosso. Query Expansion for Mixed-Script Information Retrieval. SIGIR, 2014.
3. R. Sequiera et al. Overview of FIRE-2015 Shared Task on Mixed Script Information Retrieval. FIRE, 2015.
4. A. Pirkola. The Effects of Query Structure and Dictionary Setups in Dictionary-Based Cross-Language Information Retrieval. SIGIR, 1998.
5. W. Webber, A. Moffat, J. Zobel. A Similarity Measure for Indefinite Rankings. ACM TOIS, 2010.
6. S. Robertson, H. Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. FnTIR, 2009.
7. G. V. Cormack, C. L. A. Clarke, S. Büttcher. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. SIGIR, 2009.
8. A. Shtok, O. Kurland, D. Carmel, F. Raiber, G. Markovits. Predicting Query Performance by Query-Drift Estimation. ACM TOIS, 2012.
9. L. Wang, J. Lin, D. Metzler. A Cascade Ranking Model for Efficient Ranked Retrieval. SIGIR, 2011.
10. T.-Y. Liu. Learning to Rank for Information Retrieval. FnTIR, 2009.
11. N. Reimers, I. Gurevych. Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation. EMNLP, 2020.
12. A. Abdaoui, C. Pradel, G. Sigel. Load What You Need: Smaller Versions of Multilingual BERT. SustaiNLP, 2020.
13. A. Ramanathan, D. D. Rao. A Lightweight Stemmer for Hindi. EACL Workshop on Computational Linguistics for South Asian Languages, 2003.
14. L. Wang et al. Multilingual E5 Text Embeddings: A Technical Report. arXiv, 2024.
15. C. D. Manning, P. Raghavan, H. Schütze. Introduction to Information Retrieval. Cambridge University Press, 2008.

Data: MIRACL (Apache-2.0), passage text from Wikipedia (CC BY-SA 3.0). Model: intfloat/multilingual-e5-small (MIT).

Appendix

A. Query latency per system (laptop CPU, Intel i5-12450H, no GPU)

	system	median ms	p95 ms
B0	BM25, raw whitespace tokens	0.9	26.8
B1	BM25 + normalisation + stemming	0.7	9.3
B1N	B1 without stemming	0.8	7.2
B2	B1 + Roman→Devanagari transliteration	2.8	11.5
V1	tf-idf lnc.ltc cosine	0.7	9.4
S0	B1 + classic Soundex zone	11.3	20.1
S1	B1 + Dhvani zone	5.4	12.3
S2	S1 + pooled df	7.0	16.5
S3	S2 + title zone + proximity	14.0	29.7
D0	dense e5-small (int8)	12.5	15.7
D1	dense, SCD query encoder	12.5	16.3
H1	RRF(S3, D1)	23.8	38.7
L1	learning to rank	35.5	60.8
G1	gated cascade	18.6	52.8

B. Learning-to-rank feature weights

LTR feature	weight
bm25_all	-0.069
bm25_dhvani	+0.904
bm25_title	-0.189
proximity	+0.172
tfidf	+0.273
dense	+1.363
sparse_rr	+0.619
dense_rr	+0.392
lead_passage	+0.021
log_length	+0.113
roman_query	-0.226

C. Gate feature weights

gate feature	weight
top_score	-0.038
score_gap	-0.315
nqc	+0.141
dhvani_only	-0.403
unmatched	-0.090
max_idf	+0.083
query_length	-0.165
roman_query	-0.079

D. Significance tests (paired randomisation, 10,000 permutations, nDCG@10)

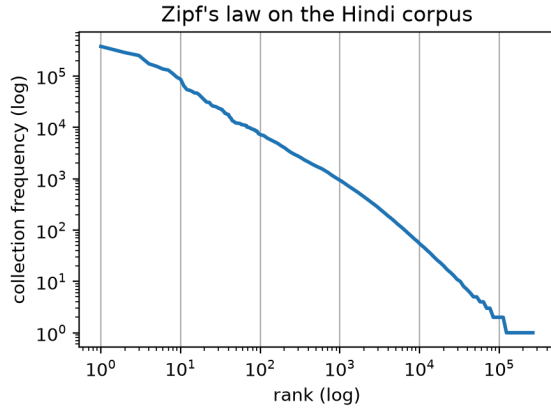
comparison	form	nDCG A	nDCG B	difference	p-value	p < 0.05
S1 vs B1	F1	0.544	0.533	+0.011	0.2645	no
S1 vs B1	F2	0.481	0.004	+0.477	0.0001	yes
S1 vs B1	F3	0.488	0.003	+0.485	0.0001	yes
S1 vs S0	F1	0.544	0.518	+0.026	0.0033	yes
S1 vs S0	F2	0.481	0.344	+0.137	0.0001	yes
S1 vs S0	F3	0.488	0.320	+0.168	0.0001	yes
S3 vs B2	F1	0.543	0.531	+0.012	0.3246	no
S3 vs B2	F2	0.488	0.113	+0.376	0.0001	yes
S3 vs B2	F3	0.491	0.095	+0.396	0.0001	yes
S2 vs S1	F1	0.533	0.544	-0.011	0.0106	yes
S2 vs S1	F2	0.487	0.481	+0.006	0.0082	yes
S2 vs S1	F3	0.490	0.488	+0.002	0.1650	no
S3 vs S2	F1	0.543	0.533	+0.009	0.2235	no
S3 vs S2	F2	0.488	0.487	+0.001	0.7658	no
S3 vs S2	F3	0.491	0.490	+0.001	0.7812	no
D1 vs D0	F1	0.598	0.601	-0.003	0.6915	no
D1 vs D0	F2	0.167	0.001	+0.166	0.0001	yes
D1 vs D0	F3	0.151	0.000	+0.151	0.0001	yes
H1 vs S3	F1	0.636	0.543	+0.093	0.0001	yes
H1 vs S3	F2	0.427	0.488	-0.061	0.0002	yes
H1 vs S3	F3	0.426	0.491	-0.065	0.0001	yes
L1 vs H1	F1	0.642	0.636	+0.006	0.5075	no
L1 vs H1	F2	0.501	0.427	+0.074	0.0001	yes
L1 vs H1	F3	0.498	0.426	+0.072	0.0001	yes
G1 vs H1	F1	0.633	0.636	-0.003	0.4160	no
G1 vs H1	F2	0.492	0.427	+0.065	0.0001	yes
G1 vs H1	F3	0.490	0.426	+0.064	0.0001	yes
B1 vs B1N	F1	0.533	0.492	+0.041	0.0003	yes
B1 vs B1N	F2	0.004	0.004	+0.000	1.0000	no
B1 vs B1N	F3	0.003	0.003	+0.000	1.0000	no

E. Efficiency structures

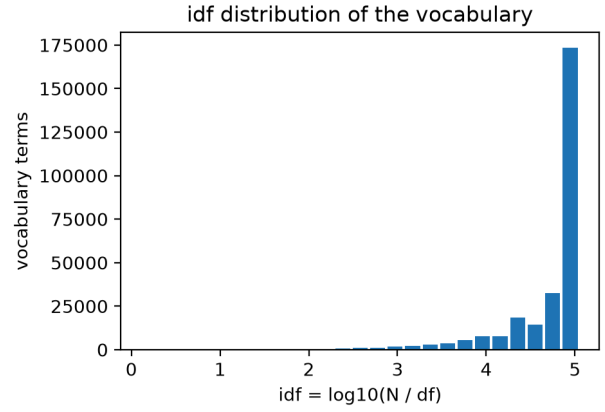
experiment	variant	median ms	nDCG@10	comparisons	docs scored	overlap@100
top-K selection	heap over scored docs (mine)	5.957	–	–	–	–
top-K selection	full sort of all N scores	1.252	–	–	–	–
champion lists (S1, r=200)	full postings	7.713	0.5045	–	–	–
champion lists (S1, r=200)	champion lists	1.282	0.3739	–	–	–
AND merge (pairs of dev query terms)	plain merge	–	–	4606	–	–
AND merge (pairs of dev query terms)	skip pointers (sqrt L)	–	–	2354	–	–
dense search (D0)	exact (all N vectors)	5.62	0.2005	–	110855	–
dense search (D0)	cluster pruning (sqrt N leaders, 8 clusters)	6.663	0.1277	–	5284	0.5024

F. idf of spelling variants on the mixed-script corpus, with and without pooled df

word	key	df Deva	df Roman	df pooled	idf Deva	idf Roman	idf pooled
यंत्र / yantra	YNTR	319	103	351	5.85	6.98	5.75
समय / samay	SMY	6,555	2,700	9,260	2.83	3.71	2.48
अधिकतम / adhikataṃ	ADKTM	331	145	492	5.81	6.64	5.42
अंतर्गत / antargat	ANTRGT	3,737	1,603	5,378	3.39	4.24	3.03
स्थित / sthit	ST	9,047	3,165	18,994	2.51	3.56	1.76
संख्या / sankhya	SNKY	2,036	807	2,918	4.00	4.92	3.64
वृद्धि / vridhhi	VRD	957	385	3,641	4.75	5.66	3.42
शक्ति / shakti	SKT	1,483	473	13,896	4.31	5.46	2.08



G. Zipf plot of the corpus.



H. idf distribution of the vocabulary.